

Rate limiting ,DDOS and Load balancer

Rate limiting = controlling how many requests a user can send to your server in a certain time.

Example:

User can send only 100 requests per minute

If they send request **#101 → server blocks it.**

Server response might be:

429 Too Many Requests

Why do we need it?

Three big reasons:

1. Prevent abuse

Example:

- Someone spams your login API **10,000 times**

Without rate limiting → server crash.

2. Protect server resources

Your server has limited:

- CPU
- Memory
- DB connections

Rate limiting keeps load **stable**.

3. Prevent brute-force attacks

Example:

Login API

POST /login

Attacker tries:

password1

password2

password3

password4

...

Rate limit:

5 login attempts per minute

Attack becomes useless.

Where is rate limiting applied?

Usually on:

API Gateway

Load balancer

Backend middleware

Example flow:

User → API Gateway → Rate Limiter → Backend Server

If limit exceeded:

Gateway returns 429

Backend never gets the request.

Rate limiting can be based on:

- **IP address**
- **User ID**
- **API key**
- **Session**

Common algorithms

There are **4 main ones**:

1 Fixed Window

Idea

Divide time into **fixed blocks (windows)** and count requests.

Example rule:

Limit: 5 requests / minute

10:00 → 10:01 5 req 10:01 → 10:02 5 req

2 Sliding Window

3 Token Bucket

4 Leaky Bucket

Problem ⚠️

Burst problem.

User can send:

10:00:59 → 5 requests

10:01:00 → 5 requests

Total:

10 requests in 1 second 😬

So **Fixed Window is inaccurate**.

2 Sliding Window (better)

Instead of resetting counter **suddenly**, we track **recent requests continuously**.

Rule:

5 requests / 60 seconds

Now user sends request:

10:01:00

System checks:

requests in last 60 seconds

If already 5 → **block**.

Pros

- ✓ More accurate
- ✓ Prevents bursts

Cons

- ✗ More memory
- ✗ More CPU

3 Token Bucket (very popular in production)

Used by companies like:

- Amazon Web Services
- Stripe

Idea

Users get **tokens**.

Each request **consumes 1 token**.

Example bucket:

capacity = 5 tokens

each req take 1 token

User must wait until **new tokens refill**.

4 Redis Distributed Rate Limiter

Now the **real production problem**.

Imagine you run **multiple servers**.

Server A

Server B

Server C

User requests can hit **any server**.

User → Load Balancer → Any server

If each server keeps **its own counter**, rate limiting breaks.

Example:

Limit = 5

User sends:

5 → Server A

5 → Server B

5 → Server C

Total:

15 requests 😬

Solution → shared store

All servers store counts in **Redis**

Architecture:

User



Load Balancer



Server A / B / C



Redis (shared counter)

Example key:

rate:user123

Value:

count = 4

Flow:

1 request

→ increment Redis counter

→ check limit

Redis operations:

INCR

EXPIRE

Example logic:

INCR user123

EXPIRE user123 60

Meaning:

count requests for 60 seconds

In node js

1 Install the library

`npm install express-rate-limit`

2 Basic Rate Limiter Setup

```
const express = require("express");
```

```
const rateLimit = require("express-rate-limit");
```

```
const app = express();
```

```
const limiter = rateLimit({
```

```
  windowMs: 60 * 1000, // 1 minute
```

```
  max: 5,           // limit each IP to 5 requests per window
```

```
  message: "Too many requests, please try again later."
```

```
});
```

```
app.use(limiter);
```

```
app.get("/", (req, res) => {  
  res.send("API working");  
});
```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

5 requests / 1 minute / per IP

5 Real production improvement

Default storage is **memory**, which is bad for multiple servers.

In production we connect it with **Redis** using **rate-limit-redis**.

Install:

```
npm install rate-limit-redis redis
```

Then:

```
const { RedisStore } = require("rate-limit-redis");  
const { createClient } = require("redis");
```

This makes the rate limiter **distributed** across servers.

The library basically does:

IP → request count

Example internal store:

192.168.1.1 → 3

192.168.1.2 → 5

When:

count > max

→ block request.

To implement rate limiting in a Node.js backend using Express.js, I usually use middleware like `express-rate-limit`. It lets us define a time window and a maximum number of requests per IP. If a client exceeds the limit, the server returns **HTTP 429 – Too Many Requests**. For production systems with multiple servers, I would store the counters in a shared store like Redis so the rate limit works consistently across all instances.

For WebSockets, the initial connection is an HTTP upgrade request and counts as a single request for normal rate limiters. After the connection is established, messages are sent over the same persistent connection, so they don't count as separate HTTP requests. To prevent abuse, servers usually implement message-level rate limiting inside the WebSocket handler, often using in-memory counters or Redis.

DDoS

DDoS (Distributed Denial of Service) is an attack where **many machines flood a server with huge numbers of requests**, making the server slow or completely unavailable for real users.

Instead of **one attacker**, the traffic comes from **thousands of compromised devices (botnet)**.

Simple flow

Botnet (1000s of machines)



Massive request traffic



Target Server



Server resources exhausted → service down

Common protections

- 1 Rate limiting** (limit requests per IP)
- 2 Load balancers**
- 3 CDN / edge protection** like Cloudflare captcha firewall
- 4 Traffic filtering and WAF**

A DDoS attack is when multiple distributed machines flood a server with massive traffic to exhaust resources and make the service unavailable. Protection usually involves rate limiting, load balancing, and edge protection services like Cloudflare.

Why people do ddos

To charge ransom because backend remain down until ddos lifted

Load balancer

A **load balancer** distributes incoming user requests across multiple servers so that **no single server gets overloaded**.

This improves:

- performance ⚡
- availability
- scalability

Simple flow

Users



Load Balancer



Server1 Server2 Server3

Instead of all traffic going to **one server**, it spreads requests across many.

Example

If **1000 users** hit your API:

Without load balancer:

1000 → Server1 ❌ overload

With load balancer:

333 → Server1

333 → Server2

334 → Server3

Servers stay stable.

Common algorithms

Load balancers decide **which server gets the request** using strategies like:

- 1 Round Robin** → requests go server1 → server2 → server3
- 2 Least Connections** → send request to server with least active users
- 3 IP Hash** → same user goes to same server

Popular load balancers

Examples used in production:

- NGINX
- HAProxy
- Amazon Web Services Elastic Load Balancer

A load balancer distributes incoming traffic across multiple servers to prevent overload and improve scalability and availability. It sits between users and backend servers and uses algorithms like round robin or least connections to decide where each request should go. Common implementations include NGINX or cloud load balancers like those from Amazon Web Services.